

Ejercicio para el laboratorio del 22S

Entrada/salida con archivos

Para operar con archivos en C++ se utilizan las clases `ifstream` (para lectura) y `ofstream` (para escritura) de la cabecera

```
#include <fstream>
```

Un archivo se abre al pasar su ruta como argumento al constructor del flujo de archivo o al usar el método `open`.

```
ifstream entrada("entrada.txt");  
ofstream salida; // archivo sin abrir  
salida.open("salida.txt");
```

Se puede comprobar que el archivo se ha abierto correctamente con el método `is_open()` y se interactúa con él como con la entrada/salida estándar.

```
int valor;  
salida << "1 + 2 = " << 1 + 2 << '\n';  
entrada >> valor;
```

Los archivos se cierran automáticamente cuando su variable sale de ámbito, pero también se pueden cerrar anticipadamente con `.close()`.

Se pide construir un programa que muestre los préstamos pendientes de una biblioteca. Se dispone de dos archivos, uno (`catalogo.txt`) con la información de los ejemplares de los que se dispone la biblioteca. El archivo comienza con un entero indicando el número total de ejemplares, que se describen cada uno en una línea con un código único, un tipo (libro, audiovisual o juego) y un nombre, separados por espacios como se muestra a continuación:

```
10  
1201 L El ingenioso hidalgo don Quijote de la Mancha  
1202 A Pulp Fiction  
1203 L Yo, robot  
1204 J Resident Evil Requiem  
...
```

Los ejemplares están ordenados de menor a mayor por su código. El otro archivo (`prestamos.txt`) contiene (en ningún orden particular) la relación de préstamos pendientes de devolución (código del ejemplar prestado, fecha de préstamo en formato AA/MM/DD e identificador numérico del usuario). El fichero también comienza con el número de préstamos:

```
7  
1204 23/09/26 223  
1212 10/09/26 2312  
1203 25/08/26 1231  
...
```

El programa comenzará cargando la información de cada archivo en las clases Catalogo y ListaPrestamos, ambas compuestas por un array dinámico de elementos de tipo Ejemplar y Prestamo, respectivamente, un contador indicando el número de elementos y la capacidad del array dinámico. El catálogo quedará ordenado por el orden numérico de los códigos (como en el archivo). Una vez leídas las listas, se ordenará la lista de préstamo por fecha de devolución. El programa permitirá mostrar la información sobre los préstamos (fecha de devolución, título, días hasta la entrega y penalización por retraso, si la hubiera) como se indica:

```
13/08/26 (en -40 días) Atraco en alta fidelidad (80 días de penalización)
16/09/26 (en -6 días) La guerra de los mundos (12 días de penalización)
23/09/26 (en 1 días) Yo, robot
06/10/26 (en 14 días) Resident Evil Requiem
...
```

Tildes en la consola de Windows

En la consola de Windows, para que las tildes, eñes y otros caracteres no ASCII se muestren correctamente es preciso incluir la cabecera `#include <windows.h>` y llamar en el main a la función `SetConsoleOutputCP(CP_UTF8)`. No escribas `using namespace std` antes de `#include <windows.h>` para evitar un conflicto de nombres.

El programa estará organizado en las clases Ejemplar, Prestamo, Catalogo y ListaPrestamo, cada una con los atributos privados y los métodos públicos que sean necesarios. Incorpora también la clase Date que se proporciona en el campus para el manejo de fechas. Cada clase se ha de implementar en sendos archivos .cpp y .h (o .hpp) separados. La clase Catalogo dispondrá de los siguientes métodos:

- Un constructor que cargue la información del flujo `istream&` que reciba como argumento. Se llamará a este constructor en el main con el flujo correspondiente al archivo `catalogo.txt`, abierto como se explica más arriba. El catálogo sólo contendrá la información de este archivo.
- Un método `buscaEjemplar` que dado un código devuelva el puntero al ejemplar (de tipo `Ejemplar`) con dicho código; si no se encuentra el código devuelve `nullptr`. Debe implementarse como búsqueda binaria¹.

La clase ListaPrestamos a su vez implementará los siguientes métodos:

- Un constructor que reciba un flujo `istream&` y una referencia constante al catálogo y cargue los préstamos desde el flujo. Se llamará a este constructor en el main con el archivo `prestamos.txt`. Cada préstamo debe incluir un campo de tipo `Ejemplar*` con el puntero al ejemplar al que hace referencia el préstamo (que será `nullptr` en caso de no encontrarse el código en el catálogo). Para obtener dicho puntero deberás llamar a la función `buscaEjemplar` del catálogo.
- El constructor llamará a un método privado `ordena` que ordenará la lista de préstamos cargada por la fecha de vencimiento de estos (anterior a posterior). Como en toda biblioteca, los ejemplares se tienen que devolver algunos días después de haber sido prestados: 30 en el caso de los libros, 7 en el de los materiales audiovisuales y 14 en el de los juegos.

¹La biblioteca estándar de C++ tiene una implementación de la búsqueda binaria en la cabecera `algorithm`, función `lower_bound`.

Ordenación en la biblioteca estándar

La cabecera `algorithm` de la biblioteca estándar incluye una función `sort` que ordena secuencias de elementos. Recibe tres parámetros: un puntero al comienzo del array, un puntero al primer elemento fuera de él y el nombre de la función que implemente el orden entre elementos. El tercer argumento es opcional y en su defecto se utilizará el operador `<` que esté definido sobre los elementos (si no hay ninguno dará un error de compilación). Puedes definir el operador `<` sobre el tipo `Prestamo` con

```
bool operator<(const Prestamo& otro) {  
    // Definición del orden  
}
```

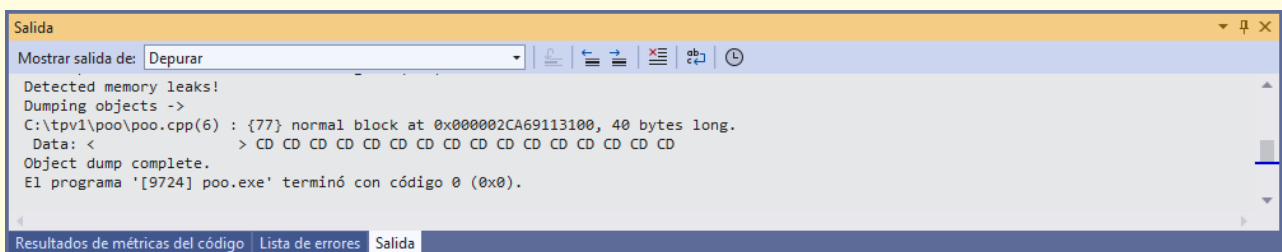
- Un método `mostrarPrestamos` o el `operator<<` que muestre la relación de préstamos con el formato mostrado en la primera página.

Además, escribe la función principal `main` en un fichero `main.cpp`. Esta debe primeramente cargar el catálogo y la lista de préstamos y a continuación mostrar un menú con opciones para mostrar el catálogo, mostrar los préstamos, prestar un ejemplar, devolver un ejemplar y salir. Al añadir un ejemplar o préstamo la lista correspondiente debe quedar ordenada, para lo cual deberá insertarse en su lugar desplazando elementos si es necesario. En caso de no haber espacio en la lista correspondiente se indicará mediante un mensaje al usuario.

En el CV puedes encontrar un ejemplo de ficheros de entrada y un fichero (`salida.txt`) con la salida que tu programa debería generar aproximadamente para dichos ficheros de entrada. Asegúrate de no dejar fugas de memoria con el archivo `checkML.h`.

Comprobación de fugas de memoria en Visual Studio

El archivo `checkML.h` disponible en el campus añade instrumentación a las funciones `new` y `delete` para detectar fugas de memoria al finalizar el programa. Has de incluir la cabecera al principio de cada archivo que reserve memoria dinámica (asegúrate también de que en la configuración del proyecto esté seleccionado el estándar C++20). Las fugas de memoria aparecerán en el panel de salida de Visual Studio.



Si usas `g++` o `clang++` en Linux o macOS esto no funcionará, pero con la opción `-fsanitize=address` el compilador mostrará también las fugas de memoria al finalizar la ejecución.

Entrega: se debe realizar una entrega por grupo en que solo debéis subir vuestro fichero fuente `.cpp` (incluyendo vuestros nombres y el identificador del grupo, si ya lo tuviérais). La entrega se realiza en el CV, en la pestaña *Laboratorio y Prácticas*.

Ejercicios adicionales: (1) crea una copia del catálogo declarando una nueva variable local de ese tipo y asignándole el catálogo original, ¿qué pasa al finalizar el programa? (2) modifica la estructura (y actualiza todo lo necesario) de manera que la estructura `Catalogo` se represente mediante un

array dinámico de punteros a objetos de tipo Ejemplar. (3) Añade una opción para añadir un nuevo material al catálogo, preguntando por consola toda la información necesaria.

Diseño de las clases: el siguiente diagrama muestra un posible diseño de las clases, con sus atributos y métodos.

